Лабораторная работа №5.

Хранение данных. Volumes и Bind Mounts

Тема: Постоянное хранение данных контейнеров и PostgreSQL в Docker

Цель: Освоить именованные Docker Volumes, Bind Mounts и подтвердить сохранность таблиц PostgreSQL после пересоздания контейнера.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №5. Хранение данных: Volumes и Bind Mounts. Продолжительность — 4 академических часа (180 минут). Среда: Debian 13 с Docker Engine; терминал Linux; интернет для скачивания образов. На примере Alpine, Nginx и PostgreSQL 18 изучаем, как сохранять данные независимо от жизненного цикла контейнера. Знания Node.js и Express.js не требуются.

2. Тема и цель

Тема: постоянное хранение файлов контейнера средствами Docker Volumes и Bind Mounts. Цель: увидеть, почему теряются файлы записываемого слоя, создать именованный том, исследовать bind mount, развернуть PostgreSQL 18 с томом в /var/lib/postgresql и подтвердить сохранность строк после удаления и пересоздания контейнера.

3. Оборудование и ПО

- Debian 13 с установленным и запущенным Docker Engine.
- Терминал с доступом к sudo.
- Подключение к Docker Hub для загрузки alpine:3.22, nginx:stable и postgres:18.
- curl для проверки Nginx (проверьте наличие заранее).
- Дополнительная учётная запись Docker Hub не требуется.

4. Схема работы

Логическая схема хранилищ



Каталог хоста и тома создаются на одном сервере Debian. Порт PostgreSQL 5432 не публикуется на хосте; простые SQL-команды выполняются через docker exec и psql. Отдельный новый том pgdata-course18 предотвращает случайное использование старых данных PostgreSQL 17. Имя контейнера оставляем pg-course для последующих лабораторных.

5. Исходные данные

Параметры лабораторной

Объект	Имя / значение

Объект	Имя / значение
Образ для проверки тома	alpine:3.22
Образ веб-сервера	nginx:stable
Образ СУБД	postgres:18
Том для файлов	lab5-files
Том СУБД	pgdata-course18
Контейнер веб-сервера	lab5-web
Контейнер СУБД	pg-course
База / пользователь	shop / student
Учебный пароль	Lab5-ChangeMe-2026
Путь тома PostgreSQL	/var/lib/postgresql
HTTP порт	8085 хоста → 80 Nginx

Пароль демонстрационный и предназначен только для изолированной учебной среды; в реальной эксплуатации не передавайте пароль открытым текстом в командной строке. Для официального postgres:18 монтируем Volume в /var/lib/postgresql, а значение PGDATA по умолчанию — /var/lib/postgresql/18/docker. Для PostgreSQL 17 путь подключения был другим (/var/lib/postgresql/data). Не переносите старый том в новую версию без процедуры обновления базы.

6. Подготовка среды

Шаг 1. Убедитесь, что Docker установлен и демон доступен.

Проверка Docker Engine

```
sudo docker version
sudo docker info
```

Ожидаемый результат: версии Client и Server, отсутствие ошибки подключения к демону. Если служба остановлена, посмотрите её состояние.

Состояние службы

```
sudo systemctl status docker
```

Если Docker остановлен, запустите службу и повторите docker version.

Запуск службы при необходимости

```
sudo systemctl start docker
```

Шаг 2. Проверка curl; не предполагаем, что программа установлена заранее.

Наличие curl

```
command -v curl
```

При отсутствии пути к исполняемому файлу установите пакет и повторите проверку.

Установка curl при необходимости

```
sudo apt update
sudo apt install -y curl
```

Шаг 3. Скачайте образы Alpine, Nginx и PostgreSQL 18. Образ postgres:18 содержит стабильную основную версию PostgreSQL 18; точная минорная версия зависит от даты загрузки.

Загрузка учебных образов

```
sudo docker pull alpine:3.22
sudo docker pull nginx:stable
sudo docker pull postgres:18
```

Проверка локальных образов

```
sudo docker image ls
```

Критерий перехода: все три образа появились в списке. Если возникли ошибки сети или реестра, сообщите преподавателю.

7. Этап 1. Данные внутри контейнера без Volume

Создайте первый временный контейнер и сохраните файл в обычной директории его записываемого слоя.

Команда создаёт контейнер из образа Alpine Linux, записывает текст в файл /tmp/check.txt, выводит содержимое файла на экран и завершает работу.

Запись данных в слой контейнера

```
sudo docker run --name lab5-temp alpine:3.22 sh -c "echo temporary > /tmp/check.txt &&
cat /tmp/check.txt"
```

Ожидается строка temporary; контейнер завершится. Убедитесь, что его можно увидеть в docker ps -a.

Контейнер после выполнения программы

```
sudo docker ps -a
```

Удалите контейнер. Данные из его записываемого слоя больше нельзя получить через этот объект.

```
Удаление временного контейнера
```

```
sudo docker rm lab5-temp
```

Создайте другой контейнер из того же образа и проверьте путь.

```

```

```
sudo docker run --rm alpine:3.22 cat /tmp/check.txt
```

Ожидается No such file or directory. Объясните, почему одинаковый образ не означает общую записываемую файловую систему контейнеров.

8. Этап 2. Создание именованного Docker Volume

Создайте том lab5-files и убедитесь в его наличии.

```
Создание и список томов
```

```
sudo docker volume create lab5-files
sudo docker volume ls
```

Ожидается строка с именем lab5-files. Укажите в отчёте имя тома и драйвер, показанный inspect.

```
Метаданные тома
```

```
sudo docker volume inspect lab5-files
```

Найдите Name, Driver и Mountpoint. Mountpoint — внутренний путь хранения Docker, не редактируйте его вручную.

Запишите файл в том из первого временного контейнера.

```
Запись в named volume
```

```
sudo docker run --rm --mount type=volume,src=lab5-files,dst=/data alpine:3.22 sh -c "echo persistent > /data/check.txt"
```

- docker run - Создать и запустить контейнер
- --rm - Автоматически удалить контейнер после завершения
- --mount - Подключить хранилище
- type=volume - Использовать Docker Volume
- src=lab5-files - Имя тома Docker
- dst=/data - Папка внутри контейнера, куда подключён том
- alpine:3.22 - Образ, из которого создаётся контейнер
- sh -c - Выполнить команду внутри контейнера
- echo persistent > /data/check.txt - Записать слово persistent в файл

Прочитайте файл из нового контейнера.

```
Проверка сохранности
```

```
sudo docker run --rm --mount type=volume,src=lab5-files,dst=/data alpine:3.22 cat
/data/check.txt
```

Ожидается persistent. Критерий перехода: новый контейнер видит файл, записанный прежним контейнером.

Проверьте доступность тома ещё раз.

```
Повторная проверка тома
sudo docker volume inspect lab5-files
```

Сравните жизненный цикл контейнера и именованного тома. Зафиксируйте наблюдения.

9. Этап 3. Bind Mount

Подготовьте каталог и HTML-файл на Debian.

```
Создание каталога на хосте
mkdir -p "$HOME/docker-lab5/site"
printf "Docker bind mount works\n" > "$HOME/docker-lab5/site/index.html"
```

Каталог и файл создаются обычным пользователем Debian; никаких системных файлов не затрагивается.

```
Проверка содержимого файла
cat "$HOME/docker-lab5/site/index.html"
```

Ожидаемый текст: Docker bind mount works.

Создайте контейнер Nginx с монтированием локального каталога в режиме readonly.

```
Запуск Nginx с Bind Mount
sudo docker run -d --name lab5-web -p 8085:80 --mount type=bind,src="$HOME/docker-
lab5/site",dst=/usr/share/nginx/html,readonly nginx:stable
```

В этой команде:

- sudo docker run - Создать и запустить контейнер
- -d - Запустить в фоновом режиме
- --name lab5-web - Назвать контейнер lab5-web
- -p 8085:80 - Связать порт 8085 Debian с портом 80 контейнера
- type=bind - Подключить обычную папку Debian
- src=$HOME/docker-lab5/site - Где находятся файлы сайта на Debian
- dst=/usr/share/nginx/html - Где Nginx увидит эти файлы внутри контейнера
- readonly - Разрешить контейнеру только чтение подкл-х файлов
- nginx:stable - Образ веб-сервера Nginx
- readonly - запрещает запись из контейнера в примонтированный каталог.

Критерий: контейнер появился в docker ps.

```
Проверка HTTP

sudo docker ps
curl http://localhost:8085
```

Ожидается Docker bind mount works. Если веб-сервер не отвечает, изучите docker logs lab5-web и список опубликованных портов.

Измените содержимое файла НА ХОСТЕ и повторите запрос.

```
Эксперимент с обновлением файла

printf "Updated by host\n" > "$HOME/docker-lab5/site/index.html"
curl http://localhost:8085
```

Ожидается Updated by host без пересборки образа и перезапуска контейнера. Сделайте скриншот двух ответов.

```
Проверка Mounts

sudo docker inspect lab5-web
```

В разделе Mounts должны присутствовать Type: bind, Source: каталог Debian, Destination: /usr/share/nginx/html, RW: false.

После проверки остановите и удалите только этот контейнер. Каталог и файл на хосте остаются.

```
Удаление контейнера и проверка каталога

sudo docker stop lab5-web
sudo docker rm lab5-web
cat "$HOME/docker-lab5/site/index.html"
```

Ожидается Updated by host после удаления Nginx.

10. Этап 4. Постоянный том PostgreSQL

Создайте новый именованный том pgdata-course18 для файлов PostgreSQL 18. Не используйте для этого упражнения том, который уже содержит кластер PostgreSQL 17.

```
Создание тома PostgreSQL

sudo docker volume create pgdata-course18
sudo docker volume ls
```

Проверьте pgdata-course18 через docker volume ls и docker volume inspect. Перед запуском убедитесь, что имя контейнера pg-course не занято старым контейнером (sudo docker ps -a). Не удаляйте старую базу данных без согласования с преподавателем.

Запустите официальный PostgreSQL 18 с именованным томом в /var/lib/postgresql и инициализационными переменными.

```
Первый запуск PostgreSQL
```

```
sudo docker run -d --name pg-course \
  --mount type=volume,src=pgdata-course18,dst=/var/lib/postgresql \
  -e POSTGRES_USER=student \
  -e POSTGRES_PASSWORD=Lab5-ChangeMe-2026 \
  -e POSTGRES_DB=shop \
  postgres:18
```

Docker подключает именованный том pgdata-course18 к /var/lib/postgresql. Сама PostgreSQL 18 использует вложенный каталог PGDATA=/var/lib/postgresql/18/docker. Переменные POSTGRES_USER, POSTGRES_PASSWORD и POSTGRES_DB применяются при первичной инициализации пустого каталога. Порт 5432 наружу не публикуется.

```
Состояние и журнал СУБД

sudo docker ps
sudo docker logs pg-course
```

При первом запуске PostgreSQL инициализирует кластер. В журнале должны появиться сообщения о готовности; проверку pg_isready при необходимости повторите после завершения инициализации.

```
Проверка готовности PostgreSQL

sudo docker exec pg-course pg_isready -U student -d shop
```

Разбор команды:

- sudo Выполнить с правами администратора
- docker exec Выполнить команду внутри работающего контейнера
- pg-course Имя контейнера PostgreSQL
- pg_isready Проверить готовность сервера PostgreSQL
- -U student Указать пользователя student
- -d shop Указать базу данных shop

Ожидается accepting connections. Далее проверьте реальный каталог PGDATA и версию сервера. Если контейнер ещё не готов, повторите проверку позднее.

Шаг проверки версии и каталога данных PostgreSQL 18. Выполните две команды внутри контейнера.

```
Проверка PGDATA и основной версии PostgreSQL

sudo docker exec pg-course printenv PGDATA
sudo docker exec pg-course psql -U student -d shop -c "SHOW server_version;"
```

Ожидается путь /var/lib/postgresql/18/docker и версия PostgreSQL 18.x. Значение PGDATA — внутренний каталог данных СУБД; Docker Volume смонтирован на родительский каталог /var/lib/postgresql. Запишите оба результата в отчёт.

11. Этап 5. Создание таблицы и записей

Создайте таблицу categories в учебной базе shop.

```
Создание таблицы categories
```

```
sudo docker exec pg-course psql -U student -d shop -c "CREATE TABLE categories (id
INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name TEXT NOT NULL);"
```

Разберём команду по частям:

- sudo docker exec — Выполнить команду внутри работающего контейнера
- pg-course — Имя контейнера PostgreSQL
- psql — Программа для работы с PostgreSQL
- -U student — Подключиться под пользователем student
- -d shop — Подключиться к базе данных shop
- -c — Выполнить указанный SQL-запрос
- CREATE TABLE categories — Создать таблицу с именем categories

Ожидается CREATE TABLE. Если таблица уже существует, вероятно, том уже использовался. Не удаляйте том с данными без разрешения преподавателя: проверьте, правильно ли выбран отдельный учебный том pgdata-course18.

Добавьте две строки.

```
Добавление тестовых категорий

sudo docker exec pg-course psql -U student -d shop -c "INSERT INTO categories (name)
VALUES ('Electronics'), ('Books');"
```

Ожидается INSERT 0 2.

Выполните запрос и сохраните результат.

```
Проверка данных

sudo docker exec pg-course psql -U student -d shop -c "SELECT id, name FROM categories
ORDER BY id;"
```

Ожидаются две категории: Electronics и Books. Если база новая, их ID будут 1 и 2. Сделайте скриншот или скопируйте текст ответа.

12. Этап 6. Проверка после пересоздания контейнера

Остановите и удалите PostgreSQL-контейнер, но не том.

```
Удаление контейнера PostgreSQL

sudo docker stop pg-course
sudo docker rm pg-course
```

Контейнер pg-course исчезнет из docker ps -a. Проверка тома должна показать, что pgdata-course18 существует.

```
Проверка сохранности тома

sudo docker volume ls
sudo docker volume inspect pgdata-course18
```

Критерий продолжения: том pgdata-course18 существует.

Пересоздайте контейнер PostgreSQL 18 с тем же именованным томом pgdata-course18 и тем же путём монтирования /var/lib/postgresql.

Пересоздание контейнера со старым томом

```
sudo docker run -d --name pg-course \
  --mount type=volume,src=pgdata-course18,dst=/var/lib/postgresql \
  -e POSTGRES_USER=student \
  -e POSTGRES_PASSWORD=Lab5-ChangeMe-2026 \
  -e POSTGRES_DB=shop \
  postgres:18
```

При обнаружении ранее созданного каталога PGDATA PostgreSQL не выполняет initdb заново. Повторная передача POSTGRES_* не изменяет существующих пользователей, паролей и таблиц.

Проверка готовности нового контейнера

```
sudo docker exec pg-course pg_isready -U student -d shop
```

После accepting connections выполните тот же запрос.

Проверка сохранности записей

```
sudo docker exec pg-course psql -U student -d shop -c "SELECT id, name FROM categories
ORDER BY id;"
```

Ожидается тот же результат: Electronics и Books с прежними id. Сравните вывод до и после удаления контейнера. Это главное доказательство сохранности данных в Docker Volume.

13. Диагностические эксперименты

Эксперимент A. Временная остановка сервиса

Остановите контейнер PostgreSQL и попытайтесь проверить готовность базы. Ожидается ошибка выполнения docker exec, потому что контейнер остановлен.

Моделирование остановки

```
sudo docker stop pg-course
sudo docker exec pg-course pg_isready -U student -d shop
```

Восстановите сервис командой docker start; затем дождитесь готовности и убедитесь в наличии категорий.

Восстановление после остановки

```
sudo docker start pg-course
sudo docker exec pg-course pg_isready -U student -d shop
sudo docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;"
```

Ожидается COUNT = 2. Если PostgreSQL ещё запускается, повторите проверку готовности.

Эксперимент Б. Проверка типа Mount

Исследуйте Mounts контейнера PostgreSQL 18 и найдите Type, Name, Destination. Отдельно проверьте PGDATA в работающем контейнере.

```
Просмотр подключённых хранилищ

sudo docker inspect pg-course
sudo docker exec pg-course printenv PGDATA
```

Ожидается Type = volume, Name = pgdata-course18, Destination = /var/lib/postgresql; значение PGDATA = /var/lib/postgresql/18/docker. Сравните с Bind Mount, ранее настроенным для Nginx.

14. Самостоятельное задание

Не удаляя таблицу categories, самостоятельно добавьте третью категорию Hardware. Затем ещё раз удалите и пересоздайте только контейнер PostgreSQL 18 с тем же томом pgdata-course18 и подтвердите сохранность трёх строк.

1. Добавить категорию Hardware через docker exec и psql.
2. Вывести SELECT до пересоздания и зафиксировать 3 строки.
3. Остановить и удалить только pg-course.
4. Пересоздать pg-course с прежним volume pgdata-course18.
5. Проверить pg_isready и повторить SELECT.
6. Зафиксировать Name и Destination из docker inspect.

Ожидается три категории после пересоздания. Не используйте docker volume rm, docker volume prune и docker system prune.

15. Итоговая проверка

```
Итоговая диагностика

sudo docker ps -a
sudo docker volume ls
sudo docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;"
```

После индивидуального задания ожидаются работающий pg-course, существующий pgdata-course18 и 3 строки в categories. Именованный том lab5-files также должен присутствовать. Контейнер lab5-web уже удалён.

16. Очистка после защиты

После защиты удалите только временные объекты, если больше не понадобятся. Контейнер pg-course и том pgdata-course18 сохраните для последующих занятий. Не удаляйте чужие Docker Volumes.

```
Удаление только временного тома

sudo docker volume rm lab5-files
```

Прежде чем удалять lab5-files, убедитесь, что он действительно не используется и его содержимое больше не нужно. Не удаляйте pgdata-course18; не выполняйте docker volume prune. Каталог $HOME/docker-lab5/site можете сохранить.

17. Что должно получиться

- Понимание различий записываемого слоя контейнера, Volume и Bind Mount.
- Созданный и исследованный именованный том lab5-files с проверкой сохранности файла.
- Проверенная связь каталога Debian с Nginx через Bind Mount.
- Запущенный контейнер PostgreSQL 18 с томом pgdata-course18.
- Таблица categories со строками, сохранившимися после удаления и пересоздания контейнера.
- Рабочий контейнер pg-course и сохранённый том pgdata-course18 для будущих лабораторных.

18. Содержание отчёта

7. Тема, цель, исходная среда и версия Docker.
8. Логическая схема Volume / Bind Mount / PostgreSQL.
9. Команды docker volume create, ls, inspect и краткое объяснение.
10. Результат эксперимента с записываемым слоем: FILE_NOT_FOUND.
11. Результат проверки файла в named volume после удаления временного контейнера.
12. Файл index.html на Debian, команда bind mount, вывод curl до и после изменения.
13. Команда запуска PostgreSQL и вывод pg_isready.
14. Вывод PGDATA и SHOW server_version для PostgreSQL 18; проверка Destination в docker inspect.
15. SQL CREATE TABLE, INSERT и SELECT до пересоздания PostgreSQL.
16. Вывод docker volume ls после docker rm pg-course.
17. SQL SELECT после пересоздания контейнера и в самостоятельном задании.
18. Результаты двух диагностических экспериментов.
19. Вывод: почему Docker Volume подходит для базы данных, а Bind Mount — для исходных файлов.

19. Контрольные вопросы

20. Где находятся изменения, если контейнер запущен без Docker Volume?
21. Что происходит с записываемым слоем после docker stop и docker rm?
22. Как работает именованный Docker Volume?
23. Чем Docker Volume отличается от Bind Mount?
24. Почему для Nginx мы указали readonly в Bind Mount?
25. Разберите --mount type=volume,src=pgdata-course18,dst=/var/lib/postgresql.
26. Чему равен PGDATA по умолчанию в официальном postgres:18?
27. Почему том PostgreSQL 18 монтируется в /var/lib/postgresql, а не в /var/lib/postgresql/data?
28. Почему нельзя без миграции подключить каталог данных PostgreSQL 17 к PostgreSQL 18?
29. Почему после пересоздания контейнера таблица categories остаётся?
30. Почему новые POSTGRES_PASSWORD и POSTGRES_DB не меняют уже созданный кластер?
31. Почему Volume не заменяет резервное копирование и какие поля docker inspect проверяют подключение тома?